Reviewer Pack — Minimal Topological Entropy of Planar Embeddings and SAT Cla…

Entry 28ff6baa8688 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 18:57:48 UTC

Minimal Topological Entropy of Planar Embeddings and SAT Clause Set Complexity

Entry ID: `28ff6baa8688` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 18:57:48 UTC

1. Conjecture

Field A (mathematical branch): Topological Dynamics **Field B** (complexity object): SAT Clause Sets

Statement:

> The minimal topological entropy (h_min) of a planar embedding of an arbitrary instance I of a satisfiability problem with m clauses is linearly correlated with its clause set complexity (c(I)), such that h_min = Θ(c(I)).

Rationale (proposer's reasoning):

> Planar embeddings can capture the structural complexity of combinatorial problems, and topological entropy measures the dynamical complexity of the system. A higher clause set complexity suggests a more intricate problem structure, which might be reflected in higher topological entropy.

Taxonomy category: `topological_entropy` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c583f789bad0a217`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal topological entropy (h_min) is linearly correlated with the clause set complexity (c(I)) if h_min / c(I) falls within [0.5, 1.5] for at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `topological dynamics AND minimal topological entropy planar embeddings` - `SAT clause sets AND complexity linear correlation minimal topological entropy` - `planar embedding satisfiability problem instance m clauses topological entropy clause set complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1208.0785v1] Estimating topological entropy from the motion of stirring rods - [http://arxiv.org/abs/2208.14597v2] A comparison of categorical and topological entropies on Weinstein manifolds - [http://arxiv.org/abs/1607.07412v3] Étale dynamical systems and topological entropy - [http://arxiv.org/abs/1908.01624v1] Learned Clause Minimization in Parallel SAT Solvers - [http://arxiv.org/abs/2207.13577v2] Scalable Proof Producing Multi-Threaded SAT Solving with Gimsatul through Sharing instead of Copying Clauses - [http://arxiv.org/abs/2005.12856v2] Topological Entropy for Arbitrary Subsets of Infinite Product Spaces - [http://arxiv.org/abs/2308.03822v1] Search for Eccentric Black Hole Coalescences during the Third Observing Run of LIGO and Virgo - [http://arxiv.org/abs/2509.08054v1] GW250114: testing Hawking's area law and the Kerr nature of black holes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_instance(m):
        variables = set()
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, 2)
            clauses.append(clause)
            variables.update(clause)
        return clauses

    def fundamental_group_size(embedding):
        # Placeholder function to simulate the computation of the fundamental group size
        # This is a dummy implementation and should be replaced with actual logic
        return len(embedding)

    def topological_entropy(fundamental_group_size):
        if fundamental_group_size <= 0:
```

```
            return 0.0
        return math.log2(fundamental_group_size + 1)

    m_values = [5, 10, 15, 20, 30, 40]
    h_min_sum = 0
    c_I_sum = 0
    instances_tested = 0

    for m in m_values:
        for _ in range(5):  # Test each size with 5 different instances
            instance = generate_random_instance(m)
            c_I = len(instance)
            embedding = instance  # Placeholder for actual embedding logic
            h_min = topological_entropy(fundamental_group_size(embedding))
            h_min_sum += h_min
            c_I_sum += c_I
            instances_tested += 1

    mean_h_min = h_min_sum / instances_tested
    mean_c_I = c_I_sum / instances_tested
    correlation = (mean_h_min * mean_c_I) / (math.sqrt(mean_h_min**2 * mean_c_I**2))

    conjecture_holds = 0.5 <= correlation <= 1.5
    counterexample = "" if conjecture_holds else f"correlation={correlation}"

    return {
        "metric_name": "Correlation between h_min and c(I)",
        "metric_value": correlation,
        "instances_tested": instances_tested,
        "n_max": max(m_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support_
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"correlation_outside_bounds\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547fe894.py", line 76, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547fe894.py", line 47, in run_trial
    instance = generate_random_instance(m)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547fe894.py", line 25, in generate_rand
    clause = random.sample(variables, 2)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence.  "
TypeError: Population must be a sequence.  For dicts or sets, use sorted(d).
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with proper error handling to ensure it completes and produces the required data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17126
2	preregistration	ollama_remote	glm4:latest	0	0	10458
3	novelty	ollama_remote	glm4:latest	0	0	8649
4	novelty	ollama_remote	glm4:latest	0	0	14352
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13497
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10104
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13960
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16677
9	judge	ollama_remote	glm4:latest	0	0	9524

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 114347 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/28ff6baa8688.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/28ff6baa8688.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/28ff6baa8688.tar.gz` (if generated)